

第 1 章 概述

【本章学习目标】

本章为整套课程的奠基章节，完全贴合时课程主线与 私有知识库+实时联网双模式 AI 智能搜索系统 项目开发逻辑，学完本章可达成完整理论与实操基础能力：

- 理解 AI 大语言模型（LLM）的核心定义、核心特征、分类体系与行业发展趋势，厘清大模型与传统搜索的本质差异
- 掌握主流云平台 MaaS 服务模式、国内主流模型生态、API_KEY 注册获取与企业级安全配置规范
- 熟练掌握 Conda 虚拟环境搭建、国内镜像配置、多环境隔离核心操作，适配本项目 Python 开发环境需求
- 掌握 VS Code/Cursor 开发工具、Jupyter Notebook 交互式编程环境的标准化配置与项目适配用法
- 精准界定 LangChain 核心定位，吃透六大核心组件、分层生态系统的层级逻辑与各层核心功能，建立全书技术框架认知
- 搭建完整可复用的项目初始开发环境，完成首个 LangChain 标准化问答链路实操，衔接后续 RAG、Agent 全项目开发
- 理解 LangChain 分层生态系统的层级逻辑与各层核心功能

本章是 LangChain 大模型智能搜索开发课程的奠基章节，系统讲解了 AI 大语言模型核心理论、云平台 MaaS 服务接入规范、Python 标准化开发环境搭建以及 LangChain 框架核心架构，构建了整套课程的前置知识与实操基础—— 本章摒弃纯理论堆砌，以新旧技术对比+环境实操配置+框架底层架构为核心主线，帮助开发者厘清大模型与传统搜索引擎的本质区别，掌握国产大模型生态接入、标准化开发环境部署、LangChain 组件与生态体系的核心逻辑，无需从零认知大模型原理、摸索环境配置、拆解框架底层逻辑，可快速具备大模型应用开发的基础理论素养与实操能力，为后续 RAG 知识库问答、智能 Agent、语义智能搜索等核心项目开发筑牢根基。

1.1 AI 大模型核心认知

1.1.1 AI 大语言模型基础认知

在学习 LangChain 框架与 AI 智能搜索开发前，需先建立大语言模型（LLM）的底层认知。传统搜索引擎仅能实现关键词检索，无法完成语义理解、知识整合、智能问答，而大语言模型的诞生彻底解决了传统检索的智能化短板，是本课程所有 AI 应用开发的核心基础。

1、从传统搜索到大模型的能力迭代

绝大多数入门开发者接触的首个 AI 工具为搜索引擎，其核心工作逻辑是关键词字符匹配，无任何语义理解能力。

搜索引擎的本质是帮你“找东西”。你用百度或谷歌搜“水稻病虫害防治”时，发生了这几件事，如表 1.1-1 所示：

表 1.1-1：搜索引擎帮助过程

步骤	发生什么
①你输入	关键词“水稻 病虫害 防治”
②搜索引擎	在全网网页里找包含这些词的页面
③它返回给你	100 万个链接列表（农业网站、论文、论坛帖子、视频……）
④你要做的事	自己点开、自己看、自己挑、自己总结

搜索引擎 = 一张超级大的“图书馆索引卡”，它告诉你“水稻病虫害”的资料在第几排第几架，但书要你自己翻，内容要你自己读，重点要你自己划。

通俗来说，搜索引擎是资料索引工具，仅能定位资料位置，无法解读、整合、输出答案。其优势为信息时效性强（能搜到昨天刚发布的虫情预警）、无内容编造问题；短板为无语义理解能力、无知识整合能力、无法适配口语化模糊提问，这也是大语言模型的核心价值所在。

2、大语言模型核心定义与通俗解读

大语言模型（Large Language Model, LLM）是依托海量文本代码数据、万亿级参数规模、大规模算力集群训练而成的深度学习模型，是自然语言处理与通用人工智能的核心基础设施，核心能力是理解、生成、推演人类自然语言与标准化代码。

通俗理解：大语言模型是经过全网海量知识训练的“智能知识整合学徒”，无需用户检索、阅读、提炼资料，接收自然语言提问后，可直接整合知识、输出完整答案，完美弥补搜索引擎“只找资料、不给答案”的核心短板。当你问“水稻病虫害怎么防治”，他凭记忆直接口述一套防治方案给你，帮你省去了“找书 + 阅读 + 提炼”的全过程。

3、大语言模型的特征

商用主流大模型具备四大核心特征，也是其拥有智能能力的核心前提，具体如表 1.1-2 所示：

表 1.1-2：大语言模型的特征

特征	说明	案例
----	----	----

参数规模庞大	参数量级从数亿至万亿级，参数越多，模型语义理解、逻辑推演能力越强	GPT-3 拥有 1750 亿参数，国产通义千问系列达万亿级参数
训练数据海量	训练依托 TB 级全网文本、论文、代码、行业文档数据，覆盖绝大多数人类知识场景	GPT-3 训练依托 45TB 高质量文本数据
算力需求极高	依赖 GPU/TPU 集群长时间迭代训练，单次完整训练成本极高，普通开发者无法本地从零训练	顶级大模型单次训练周期可达数月，成本超千万美元
涌现能力	参数、数据、算力达到临界值后，自主衍生出翻译、推理、编程、创作等未专项训练的高阶能力	基础文本模型自主掌握代码调试、数学推理、多语言翻译能力

4、大语言模型的分类

主流大模型可通过模态和核心功能两大维度分类，适配不同 AI 智能搜索、问答、创作场景：

（1）按模态划分：适配输入输出场景。分类如表 1.1-3 所示：

表 1.1-3：按模态划分的大语言模型分类

类型	说明	代表模型
纯文本大语言模型	仅支持文本输入输出，专注语义理解、问答、文本创作、代码编写，适配本课程智能搜索开发	GPT-3.5、通义千问、DeepSeek 系列
多模态模型	支持文本、图像、音频、视频多类型输入输出，可实现图文问答、图像理解、视频内容解析	GPT-4、文心一言 4.0、通义千问-VL
视觉大模型	专注图像生成、图像识别、视觉推理，无文本深度问答优势	通义万相、Stable Diffusion、Sora

（2）按功能划分：适配开发需求。分类如表 1.1-4 所示：

表 1.1-4：按功能划分的大语言模型分类

类型	核心用途	代表模型
生成式模型	专注文本、图像、音频、视频内容创作	文案撰写、素材生成、多媒体创作
推理模型	强化数学计算、逻辑推理、代码调试、复杂问题拆解能力	代码开发、算法解题、技术问答
具身智能模型	适配工业控制、机器人自动化、实景场景交互	智能制造、实景设备调度

5、大模型行业发展趋势

当前大模型行业已从通用能力比拼转向工程落地、垂直适配、开源普惠，四大核心趋势对开发者至关重要：

（1）国内外性能差距持续缩小：国产开源模型（DeepSeek、通义千问、GLM）快速迭代，通用能力与海外顶尖模型差距大幅收窄，完全满足中小企业开发落地需求。

(2) 垂直场景深度渗透：从通用对话逐步落地到农业、医疗、教育、工业、政务等垂直领域，私有化、行业定制化开发成为主流。

(3) 多模态融合常态化：单一文本模型逐步迭代为图文音视频一体化模型，智能搜索从文本检索升级为全模态检索问答。

(4) 开源生态全面繁荣：大量优质模型开源免费，搭配云平台 MaaS 服务，大幅降低个人开发者、中小企业的落地门槛。

6、大语言模型底层工作原理与核心边界

所有商用大模型均为概率式文本生成机制，无人主观意识与真实理解能力，仅通过海量数据规律复刻实现智能效果，可分为三层核心逻辑：

1) 底层核心机制：纯概率预测，无真实语义理解

大模型从根源上不会“读懂”文字含义，所有输出都是基于海量数据的概率计算结果。

(1) 模型训练本质

模型训练阶段会学习全网海量文本、书籍、论文、开源代码、行业文档等一切公开资料，持续提取文字、词语、句子、代码片段的共现规律，并将所有语言信息转化为数字向量存储。在对话、生成内容、编写代码的过程中，模型不会解读语句、需求、业务逻辑的内在含义，只会基于用户输入的上文内容，计算所有文字、词汇、代码片段的出现概率分布，最终优先选择高概率内容逐字、逐段接续生成。

通俗的讲：模型不会思考“这句话是什么意思、这个需求要实现什么效果”，只会精准计算“人类常规行文、编程、表达逻辑中，当前上下文之后，下一个字、下一段代码最可能出现的内容是什么”。

(2) 与老式文本生成的核心区别

早期传统 AI 文本生成是碎片化随机拼接，无序堆砌文本片段，语句不通、逻辑混乱；而当前主流大模型基于 Transformer 架构，搭载全局注意力机制，可一次性读取上万字的全文上下文，全局权衡所有文字、代码的衔接概率，并非简单摘抄局部短句、片段，因此输出内容连贯、规整、符合人类表达习惯。

例如续写句子，你输入：本年度片区水稻遭遇初夏持续高温热害，植株抽穗扬花期受阻，花粉活性大幅下降，田间出现大面积空壳、瘪粒现象。农技监测数据显示，当前稻田根系发育偏弱、土层保水保肥能力下降，现有补救方案仅能通过分期喷施叶面肥、适度浅灌水降温，缓解植株衰败态势，最大程度降低减产损失，但需避开正午高温、暴雨天气，严格把控水肥用量……

需要续写下一段技术分析内容。

①老式模型（RNN/LSTM、早期碎片拼接模型）

逻辑缺陷：只能看近处几十字，抓不住全局农业技术线索

它仅聚焦最近一两句话：叶面肥、浅灌水、规避恶劣天气，完全遗忘前文关键信息：高温热害、花粉失活、空壳瘪粒、根系偏弱、保水保肥差、降低减产损失。

生成结果（逻辑割裂、局部拼凑）：稻田需要喷施叶面肥，及时灌水保湿，避开雨天和高温天气，做好田间养护，改善水稻生长状态。

问题：

- 完全忘了核心种植问题是高温热害导致扬花受阻、籽粒空瘪，无针对性技术逻辑；
- 重复堆砌“叶面肥、灌水、避天气”局部词语，只是农业词汇随机拼接碎片；
- 前后没有因果：水肥调控的核心目的是缓解热害、减少减产，模型完全没关联这

条长线技术逻辑；

- 文本只是基础农事操作罗列，没有完整的田间补救技术脉络。

②Transformer 大模型（依靠全局自注意力）

- 逻辑优势：一次性读取全部 3000 字农业前文，全局计算每个字词关联权重
- 自注意力机制会建立全部远距离农业技术绑定关系：
 - 高温热害—扬花受阻、花粉失活；
 - 根系偏弱、保水保肥差—限定浅灌、精准补肥；
 - 叶面肥+调水操作—缓解植株衰败、降低减产损失；
 - 恶劣天气—农事操作禁忌约束。

所有农技条件、核心痛点、操作约束、补救目标同时参与下一句概率计算，不是只看临近短句。

生成结果（全局逻辑连贯，长线技术脉络统一）：本次田间水肥调控需严格贴合高温热害后的水稻生长特性，鉴于植株根系活力弱、土层保肥性不足，切忌大水漫灌和过量追肥，避免加重根系负担。工作人员需选择早晚低温时段作业，精准配比叶面肥浓度，通过浅灌水降温、叶面补营养的双重方式，改善水稻植株衰败状态，弥补扬花期受损缺陷，最大限度减少空壳瘪粒带来的减产影响。

直观体现注意力全局权衡的细节：

- 主动关联开篇长线农技伏笔：高温热害、花粉失活、根系偏弱，技术补救逻辑完整闭环；
- 把“水肥调控、叶面补肥”和核心目标“缓解热害损伤、降低减产”深度绑定，不是孤立罗列农事操作；
- 将天气禁忌、植株长势短板转化为操作标准，规避错误农事行为，贴合整体农技方案；
- 全文技术逻辑、操作规范、补救目标和几千字前文的田间监测数据、病害背景完全统一，无技术矛盾和逻辑断层。

老式序列模型：窗口极小，只能抓取相邻短句，生成等于农业词汇碎片随机拼接，处理长篇农技方案、作物长势分析、灾害补救策略时，极易丢失前置病害背景、植株状态、操作禁忌、核心增产目标等长线信息，技术逻辑断裂、方案片面失准。

Transformer 自注意力：全局扫描全文所有农业文本，让远距离的病害成因、植株性状、农事操作、约束条件、生产目标互相建立关联，生成每一段农技内容时都综合全文上下文权衡，长线种植逻辑、深层技术因果、完整补救方案不会丢失。

2）表层表现：看似理解，实则是规律复刻与涌现

大模型之所以能写出逻辑通顺、内容深刻、贴合需求的文本与代码，并非拥有思考能力，而是完整复刻了人类的知识规律与思维范式。

（1）文本统计规律复刻人类全部逻辑与常识

人类的知识体系、因果逻辑、写作范式、专业规则、修辞表达，全部隐藏在海量训练语料的文字关联规律中：

- 议论文创作：模型学习千万篇范文固定的“论点-论据-总结”结构，依靠概率自动复刻规行文逻辑；
- 专业问答输出：教材、论文统一的“概念-解释-案例”表达范式，被模型完整捕捉并复用；

- 文学与情感表达：亿万文学作品中的比喻、排比、氛围感句式，会被模型优先调用，模拟人文共情效果；
- 代码编写：海量开源项目、编程文档的语法规则、函数调用、项目架构范式，成为模型代码生成的概率依据。

当训练数据覆盖人类绝大多数思考、表达、创作、开发场景，模型的概率生成结果，就会完美模拟出理解、推理、共情、专业输出的外在效果。

（2）涌现能力：概率模型衍生出近似思考的高阶能力

大模型存在核心的涌现现象：当模型参数规模、训练数据量、算力资源达到临界值后，原本单纯的逐字、逐段概率预测机制，会突然衍生出逻辑推导、归纳总结、辩证分析、多角度创作、复杂代码拆解等高阶能力。

例如让 AI 撰写低碳发展主题议论文，模型不会随机拼凑语句，而是从海量政策文件、社科文献中习得完整因果链条：碳排放危害—双碳政策—产业转型—民生平衡—未来展望，整套严谨的逻辑链会在概率计算中自然串联成型，最终产出结构完整、逻辑通顺、具备深度的文章。

3）核心边界：模拟理解 $\neq$ 人类真实理解

大模型具备极强的模拟认知能力，但完全不具备人类的主观认知与真实理解，二者有本质边界。

（1）模型拥有的模拟能力

- 语义匹配：精准识别提问、需求中的关键词与内在逻辑关系；
- 逻辑推演模仿：复刻人类常规思维、推导范式输出结论；
- 知识整合与创作：整合海量碎片化知识，完成文本润色、内容创作、修辞优化；
- 长文本全局统筹：依托全局注意力机制，产出结构完整、逻辑流畅的长文与工程代码。

（2）模型完全缺失的人类核心认知

- 无主观意识：无法感知文字、内容对应的真实情绪，书写喜怒哀乐、利弊评价时，没有任何真实感受；
- 无现实体验：从未接触真实物理世界，对万物的认知仅停留在文字、代码描述层面，无实体感知；
- 无自主领悟能力：所有因果逻辑、专业道理、开发规则均是复刻语料内容，并非自身实践、思考总结所得；
- 存在概率幻觉：仅依据高概率生成内容，不会自主核验事实真伪，极易产出看似合理、实则完全虚假的文字、数据、代码函数。

总之，当前主流的纯自回归概率生成技术路线，无法诞生真正的自主理解与人工智能意识，大模型本质只是一款高精度语言模拟器。目前行业正在探索多模态大模型、世界模型、具身智能等新型技术方向，尝试让模型建立对现实世界的实体认知与真实理解，但现阶段所有商用落地的大模型产品，依旧全部停留在概率文本/代码生成阶段。

7、AI 编程的底层逻辑

AI 编程依托大模型实现代码生成、程序开发、项目迭代，本质是人机协同的代码生成与迭代过程，并非 AI 自主完成全流程开发，其核心逻辑与大模型概率式文本生成机制完全统一。

1）底层原理

AI 编程专用大模型，预训练于海量开源代码、编程文档、工程项目代码库，学习各类编程语言的语法规则、函数调用关系、项目架构范式、代码调试规律。接收用户自然语言开发需求后，模型不会自主设计技术方案，仅通过计算后续代码片段的概率分布，优先输出符合编程规范、上下文逻辑通顺的高概率代码，核心是“预测最优代码片段”，而非人类式的主动设计与思考。

2) 标准工作流程

①需求转译：将用户自然语言功能需求，依托训练得到的编程范式，拆解为模块化代码并生成初始版本；

②迭代调试：初始代码普遍存在依赖缺失、逻辑 bug、语法错误等问题，用户通过自然语言反馈报错信息，模型结合全文上下文迭代修正代码，多轮交互后得到可运行程序；

③工程补全：Cursor 等 AI 专用 IDE 可读取完整项目代码库，自动补全注释、统一代码风格、排查跨文件逻辑冲突，完成工程化优化。

3) 能力边界

(1) 优势：标准化样板代码、简单功能脚本、通用工具代码生成效率极高，可大幅减少程序员重复性编码工作；

(2) 短板：复杂系统架构设计、小众非标准技术栈适配、核心业务逻辑深度优化，必须由人类主导把控；模型生成代码易暗藏安全漏洞、出现幻觉式无效函数调用，所有 AI 生成代码必须经过人工审核、测试校验后方可上线。

8、大语言模型和搜索引擎

1) 大语言模型和搜索引擎的区别

大语言模型和搜索引擎的区别如表 1.1-5 所示

表 1.1-5：大语言模型和搜索引擎的区别

对比维度	搜索引擎	大语言模型
核心能力	检索已有内容，仅做信息匹配	整合生成全新内容，做知识提炼与解答
输入形式	精准关键词短句。如关键词“水稻 病虫害 防治”	完整自然语言提问、模糊口语化需求，如完整句子“水稻病虫害怎么防治”
输出结果	原始网页链接、碎片化文本（指向别人写的内容）	结构化、完整的专属答案（自己现写的）
信息时效性	实时更新，可查询最新资讯数据，能搜到昨天刚发布的虫情预警	受训练时间限制，存在知识滞后性，即只记得训练截止前的内容
内容真实性	无编造能力，仅转发公开内容	存在幻觉，可能编造虚假信息，为了文字通顺可能添油加醋
溯源能力	能，点开链接看原始出处，可直接跳转原始来源核验	不能，它不记得“这句话从哪本书来的”，无精准溯源能力，无法标注信息出处

搜索引擎是“找东西”，大语言模型是“造东西”。

2) 大语言模型能做什么

大语言模型具有搜索引擎做不到的能力。如表 1.1-6 所示：

表 1.1-6：大语言具备的能力

能力	案例
直接给答案	不用点开 100 个农业网站自己对比总结
改写润色	把“地干了，赶紧浇水”改成“土壤墒情不足，建议及时补水灌溉”
总结提炼	把 3000 字的玉米高产论文压成 5 条核心措施
头脑风暴	“帮我起 10 个带土字的生态农场名字”
角色扮演	假装植保专家、农技推广员来回答你
翻译转换	把中文农业技术手册翻译成英文，或把需求翻译成 Python 代码

凡是跟“组织语言”有关的事，大模型都极其擅长。

3) 大语言模型不能做什么

大语言模型弱在哪（使用时必须警惕），如表 1.1-7 所示：

表 1.1-7：大语言不具备的能力

短板	案例	应对策略
会胡说八道 (幻觉)	问“月球种水稻怎么搞”，它编“用陨石坑蓄水、嫦娥负责插秧”	重要事实必须交叉验证
知识有时效性	不知道昨天刚发布的病虫害预警	开启联网搜索功能
不能查来源	你不知道它的话是从哪篇论文里拼出来的	要求它提供依据，或自行核对
缺乏真实体验	它没真的下过地，不懂“墒情”“倒春寒”的真实感受	结合实地经验判断
算术不靠谱	问“每亩用 200 毫升药兑 30 升水，3 亩需要多少药和水”	用代码执行器或计算器工具

大模型是“语言大师”，不是“领域专家”。用它做初稿，靠自己做核实。

总之，搜索引擎擅长找实时事实，大模型擅长懂语义、整答案、做创作。

1.1.2 云平台 MaaS 服务与模型接入

普通开发者无法本地训练大模型，行业通用落地方式为 MaaS（模型即服务），通过云平台 API 接口快速调用各类大模型，无需 GPU 算力、无需复杂部署。

1、MaaS 三种标准化接入模式

主流云厂商均提供 MaaS（模型即服务）能力，支持 API 直连、一键部署、定制微调三种接入模式，适配快速验证到企业级生产的全场景。云服务平台接入大模型已高度标准化，开发者可根据需求选择合适模式，快速实现 AI 能力集成。如表 1.1-8 所示：

表 1.1-8：MaaS 的三种接入模式

接入模式	适用场景
API 直连	快速验证、原型开发
一键部署	企业级生产环境，保障数据安全
定制微调	针对垂直领域（如农业、医疗）进行模型优化

2、国内主流适配云平台

优先选用国内可稳定访问、支持 OpenAI 兼容接口、提供免费额度的平台，规避境外网络访问问题。主流云平台模型生态如表 1.1-9 所示：

表 1.1-9：主流云平台模型生态

云平台	模型生态亮点	官网入口
阿里百炼	通义千问、DeepSeek、GLM-4.5	https://bailian.console.aliyun.com
DeepSeek	DeepSeek-V3、DeepSeek-R1	https://www.deepseek.com
硅基流动	通义千问、DeepSeek、GLM 等	https://www.siliconflow.cn
火山引擎	豆包大模型系列（语言、语音、图像、视频生成）	https://www.volcengine.com/
腾讯云	腾讯混元大模型及第三方模型	https://cloud.tencent.com/
百度智能云	文心大模型系列（ERNIE）、第三方开源模型	https://cloud.baidu.com/
华为云	盘古大模型、昇腾云开源大模型（含 DeepSeek 等）	https://www.huaweicloud.com/

3、API_KEY 企业级安全配置规范

API_KEY 是云服务唯一身份凭证，严禁硬编码写入代码、严禁上传 GitHub、严禁公开分享。课程统一采用环境变量配置方案，保障项目安全性与可移植性：

- 核心规范：在项目根目录创建.env 文件，存储 API_KEY、BASE_URL 等敏感信息
- 读取方式：通过 python-dotenv 库加载环境变量，代码中动态读取
- 安全拓展：项目上线前将.env 加入.gitignore，避免敏感信息泄露

1.2 项目开发环境标准化搭建

本课程所有代码、案例、项目均基于统一环境开发，本章完成全套环境搭建，后续所有单元无需重复配置，实现一次搭建、全程复用。

1.2.1 conda 虚拟环境管理

Conda 是一个开源的跨平台包管理和环境管理系统，主要用于解决软件包依赖冲突和多版本管理问题。

1、核心功能

- 包管理：安装、更新和卸载软件包，自动处理依赖关系。
- 环境管理：创建隔离的虚拟环境，避免不同项目间的依赖冲突。
- 跨平台支持：在 Windows、macOS 和 Linux 上一致运行。

2、与 pip 的区别

Conda 与 pip 的区别如表 1.2-1 所示：

表 1.2-1：Conda 与 pip 区别

对比维度	pip	Conda
管理范围	仅管理 Python 第三方包	支持 Python、C++、R 等多语言包
依赖处理	依赖处理能力弱，易出现版本冲突	自动解析、适配所有依赖，彻底规避冲突
环境隔离	需配合 venv 使用，隔离性弱	原生支持虚拟环境，完全隔离多项目
适用场景	简单小型 Python 项目	AI 复杂项目、多依赖工程化项目（本课程选用）

2、安装选型

- Anaconda：包含 Conda 和大量数据科学库的完整发行版。新手和数据科学用户安装 Anaconda（约 3GB）。
- Miniconda：轻量版，仅包含 Conda 和 Python，适合自定义安装。熟练开发者安装 Miniconda（约 100MB）。

3、Anaconda 安装

1) 下载安装包

- 官网下载：在官方网站下载。
- 支持 Windows 10/11、macOS 10.13+、Linux（Ubuntu/CentOS 等）。
- 根据系统选择对应安装包（如 Windows 选.exe，macOS 选.pkg，Linux 选.sh）。

2) Windows 系统安装步骤

运行安装程序，根据安装向导点击“Next”>“I Agree”接受许可协议。

（1）安装类型：

- All Users（需管理员权限，适合多用户电脑）。
- Just Me（仅当前用户，推荐个人使用）。

（2）安装路径：建议默认路径（如 d:\Program Files\anaconda3），避免中文。

（3）关键勾选：Add Anaconda3 to my PATH environment variable（添加环境变量，避免后续手动配置）

（4）完成安装：安装成功后点击 Close，不勾选 Learn More about Anaconda 等附加选

项。

(5) 验证安装：打开终端（cmd），输入：

```
conda -V
```

显示版本号（如 conda 25.x.x）即成功。

(6) 配置国内镜像（加速下载）

课程统一使用清华镜像源，终端执行以下命令完成配置，全程生效：

```
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free
conda config --set show_channel_urls yes
```

验证配置：

```
conda config --show channels
```

4、常用命令速查

Conda 常用命令如表 1.2-2 所示：

表 1.2-2：Conda 与 pip 区别

命令	功能
conda create -n 环境名 python=版本号	创建环境
conda activate 环境名	激活/进入环境
conda env list	查看所有环境
conda deactivate	退出当前环境
conda env remove -n 环境名	删除环境
conda install -c conda-forge 包名	从 conda-forge 渠道安装包

5、conda 环境创建

对于不同项目有不同的需求，如果在同一个环境下运行这些项目，很可能会出现某些包的版本冲突等问题。因此需要虚拟环境来隔离不同的项目，以避免环境冲突。conda 支持多环境隔离，用户可通过命令行实现环境管理，常用命令包括创建环境、切换环境等。

创建 AI 开发环境示例：

```
# 创建名为 ai_env、Python 3.13 的环境
conda create -n ai_env python=3.13 -y

# 激活环境
conda activate ai_env

# 验证 Python 版本
python --version
```

1.2.2 开发工具标准化配置

1、VS Code 核心配置

VS Code 是微软推出的免费开源代码编辑器，凭借轻量高效、插件生态丰富成为全球开发者首选工具。

1) 核心特点：

- 完全免费开源，无功能阉割
- 支持 Windows、macOS、Linux 全平台

- 内置 Git 集成、调试器、终端
 - 插件生态极其丰富（数万款扩展）
- 2) AI 助手说明：
- VS Code 内置的 GitHub Copilot 分为免费版（有限额）和付费版
- 3) 首次启动必做配置

首次启动必做配置步骤如表 1. 2-3 所示：

表 1. 2-3：首次启动必做配置步骤

步骤	操作
①安装中文扩展	Ctrl+Shift+X → 搜索“Chinese (Simplified)” → 安装后重启
②切换主题	Ctrl+Shift+P → 输入“颜色主题” → 选择喜欢的主题
③安装通义灵码	Ctrl+Shift+X → 搜索“Qoder” → 安装 Qoder CN 扩展 → 登录
④安装 Python 支持	Ctrl+Shift+X → 搜索 Python → 安装由 Microsoft 发布的官方 Python 扩展（含代码补全、调试、Jupyter 支持）
⑤选择 Python 解释器	Ctrl+Shift+P → “Python: Select Interpreter” → 选择 ai_env(必须先创建 conda 环境)

- 4) 创建项目：
- (1) 新建项目：新建一个空文件夹（如 my-demo-project）
 - (2) 打开项目：VS Code 中选择菜单文件 → 打开文件夹 → 选择该文件夹
 - (3) 新建文件：左侧空白处右键 → 新建文件（如 main.py 或 test.ipynb）

5) GitHub Copilot Chat

Copilot Chat 大家俗称 “VS Code 自带 AI 聊天”，使用时需要登录 GitHub 账号。它面向代码开发的对话式 AI 助手，依托云端大模型（Claude），自动读取当前文件、选中代码、报错、终端日志，支持：

- 自然语言提问：解释代码、修复 Bug、写函数 / 接口
 - 编辑指令：重构、优化、加注释、单元测试
 - 斜杠快捷命令： /fix /explain /doc /test
 - Agent：项目级分析、多文件检索、终端命令生成
- (1) 两个易混淆概念区分
- 行内代码补全（Inline Completion）：敲代码弹出灰色提示，Tab 一键接收，独立额度；
 - Copilot Chat 对话窗口：侧边聊天面板、Ctrl+I 唤起弹窗，单独占用月度聊天额度；

(2) 免费版（Copilot Free）官方固定额度（2026 现行标准）

仅需免费 GitHub 账号，无需信用卡、无需试用，永久免费分层限制，每月 1 号自动重置额度：

代码行内补全：2000 次 / 月。输入代码弹出的灰色建议，按 Tab 确认才算消耗 1 次，只预览不 Tab 不计次数。补全 2000 次用完后不再弹出代码提示，到下月刷新。Copilot Chat 对话请求：50 次 / 月。每发送 1 轮提问 = 1 次额度；多轮连续追问、

Agent 复杂任务统一扣减聊天次数。聊天 50 次用完后 Chat 面板禁用，等到下月刷新。

2、Cusor AI 编辑器

Cursor 是基于 VS Code 深度定制的 AI 代码编辑器，核心升级为 Agent 智能体工作流、Composer 多文件编辑、Background Agents 后台自治、MCP 应用生态，全面从 “代码补全” 升级为 “AI 驱动开发平台”。 适合项目高阶迭代开发，可作为备选开发工具。

Cursor 随着版本的更新，免费功能被功能阉割，AI 助手免费限额减少。

1.2.3 Jupyter NoteBook 交互式编程环境

Jupyter Notebook 是一个基于 Web 的交互式编程环境，广泛应用于数据科学、机器学习、学术研究等领域。

1、核心特点

- 交互式编程：代码可实时执行并显示结果（图表、表格等）
- 富文档混合：支持 Markdown 文本、LaTeX 公式、HTML 和多媒体内容

2、安装方式

- 使用 Anaconda：已预装 Jupyter，无需额外安装
- 使用 VS Code：创建 .ipynb 文件即可使用（已集成 Jupyter）
- 手动安装：conda install -c conda-forge jupyter

3、核心操作快捷键

核心操作快捷键如表 1.2-4 所示：

表 1.2-4：Jupyter 核心操作快捷键

快捷键	功能
Y	切换为代码单元格
M	切换为 Markdown 单元格
DD	删除当前单元格
Ctrl+Enter / Shift+Enter	仅运行当前单元格， Shift+Enter 执行同时创建新单元格
Shift+L	显示/隐藏行号

4、Markdown

在 Jupyter Notebook 中，使用 Markdown 单元格添加注释说明是提升代码可读性和文档完整性的关键方法。

(1) 添加 Markdown 单元格：先选中已有单元格（蓝色边框），按快捷键 M。

(2) 编辑 Markdown 内容：

- 标题层级注释语法：用 # 表示标题，层级从 #（一级标题）到 #####（六级标题），清晰划分文档结构
- 普通文本：直接输入
- 列表：在文本前加一个减号

例如：

数据预处理步骤

本步骤对原始水稻产量数据进行清洗，包括：

- 删除缺失值超过 30%的列
- 对异常值进行截尾处理

按 Shift+Enter 渲染后，会显示为格式化的标题和列表，如图 1.2-1 所示：

数据预处理步骤

本步骤对原始水稻产量数据进行清洗，包括：

- 删除缺失值超过30%的列
- 对异常值进行截尾处理

图 1.2-1：Markdown 文本

(3) 最佳实践

- 代码与注释分离：每个代码块上方添加 Markdown 注释，避免在代码中写冗长注释（代码内仅保留简短单行注释 #）。
- 折叠单元格：点击单元格左侧的箭头（或按 Shift+0）折叠 Markdown 注释，聚焦代码编辑。

1.3 LangChain 大模型应用开发框架

LangChain 是本课程唯一核心开发框架，所有智能搜索、RAG 问答、Agent 自主决策功能均基于该框架实现，本章建立全局架构认知，后续单元逐一拆解落地。

1.3.1 核心定义

LangChain 是基于 Python 的开源大模型应用工程化编排框架，核心定位是连接大模型、私有数据、外部工具、记忆能力的桥梁。核心价值是封装通用开发逻辑，避免重复造轮子，让开发者快速搭建企业级大模型应用。

LangChain 就像“大模型应用的乐高积木”——把大模型、数据、工具、记忆等核心能力封装成可复用的模块，你只需拼搭这些模块，就能快速实现复杂功能。

典型应用场景：

- 智能问答系统（基于私有知识库）
- 多轮对话机器人（带记忆功能）
- 工具调用代理（调用计算器、API、数据库）
- RAG 检索增强生成（结合外部文档回答）

核心价值：

- 统一大模型调用接口：无论调用阿里百炼、OpenAI 还是 DeepSeek，都用同一套代码写法，切换模型只需改一行配置。
- 避免重复造轮子：封装了对话记忆、提示词管理、工具调用、数据连接等通用能力，开发者只需专注业务逻辑。
- 模块化+可扩展：所有功能拆分为独立模块，可按需组合。比如只用 Memory 模块增强问答，或组合全部模块实现智能代理。
- 生态兼容极广：支持几乎所有主流大模型（阿里百炼、OpenAI、通义千问、DeepSeek 等）、工具（Python 函数、REST API、数据库、代码解释器等）、数据源（PDF、Excel、知识库、向量库等）。

1.3.2 六大核心组件

LangChain 所有高阶功能均由六大基础组件组合而成，各组件职责独立、模块化可复用。具体组成如图 1.3-1 所示：

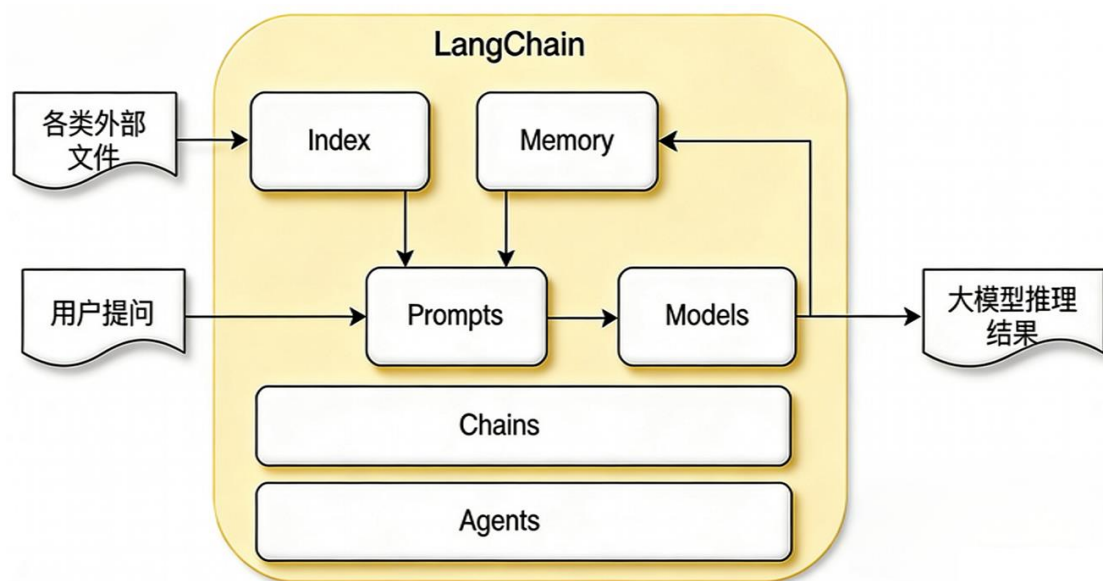


图 1.3-1：LangChain 基本构成

- (1) Models（模型）：负责处理自然语言输入并生成响应，是整个系统的“大脑”。如 OpenAI 的 GPT 模型、阿里百炼的通义千问模型、DeepSeek V3.2 等。
- (2) Prompts（提示词）：为模型提供输入指令，引导模型生成特定类型的响应。如用户输入的问题或指令，如“解释量子计算”。
- (3) Indexes（索引）：用于存储和检索外部数据，支持模型访问知识库或数据库。如向量数据库（如 Pinecone）、关系型数据库（如 PostgreSQL）。
- (4) Memory（记忆）：存储对话历史或上下文信息，使模型能够记住之前的交互。如对话历史记录、用户偏好设置。
- (5) Chains（链）：将多个组件（如模型、提示词、索引）串联起来，形成一个完整的处理流程。如从用户输入到模型响应的完整流程。
- (6) Agents（代理）：负责协调和管理其他组件，根据任务需求动态选择和调用合适的组件。如任务调度器、工作流管理器。

1.3.2 分层生态系统

LangChain 的分层生态系统是其“模块化、可解耦、按需扩展”设计理念的核心体现，整体遵循底层抽象 → 中间集成 → 上层应用 → 部署运维的逻辑，覆盖从“框架开发”到“应用上线”的全生命周期，同时适配不同角色开发者（框架贡献者、集成开发者、应用开发者、运维人员）的需求。LangChain 分层生态系统如图所 1.3-2 示：

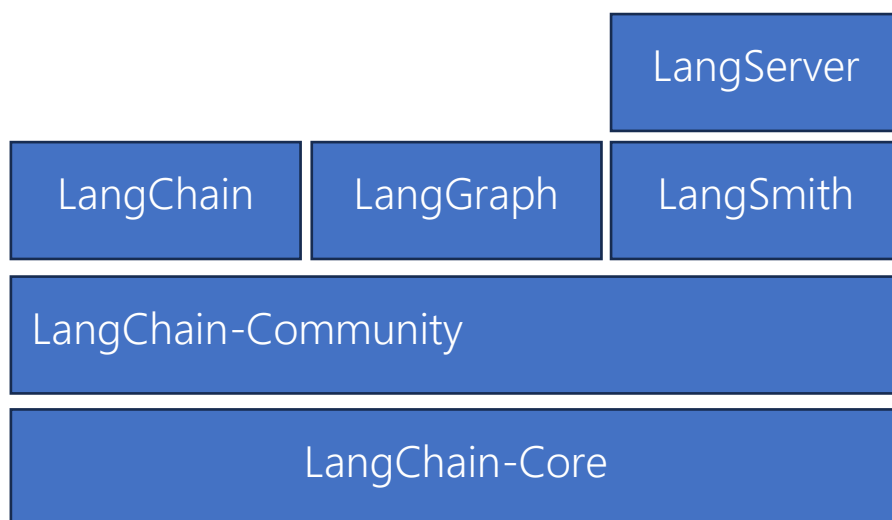


图 1.3-2: LangChain 分层生态系统

1、各层详解

1) 核心抽象层 LangChain Core（最底层）

LangChain 生态的“地基”，定义所有组件的标准化接口与数据结构，实现功能解耦，保障所有模型、工具、数据源可无缝切换、兼容适配。不包含任何具体的第三方集成，仅提供标准化的“骨架”。

(1) 核心作用

- 统一全生态的核心接口：比如 `Runnable`（所有可执行组件的基类，`.invoke()/.stream()/.batch()` 方法都来自这个抽象）、`BaseLanguageModel`（所有大模型的基类）、`BasePromptTemplate`（提示词模板基类）；
- 提供基础数据结构：比如 `Message`（`SystemMessage/HumanMessage` 等）、`Document`（数据接入层的文档结构）；
- 解耦上层逻辑：无论集成哪个大模型 / 工具，都遵循 Core 层的接口，切换集成时无需修改核心代码。

(2) 关键包安装

```
conda install -c conda-forge langchain-core
```

注意：

①安装包时激活 `ai_env` 环境。

```
conda activate ai_env
```

②如果安装 langchain 包：

```
conda install -c conda-forge langchain
```

包含了：langsmith, **langchain-core**, langchain-text-splitters, langchain。

2) 社区集成层 LangChain Community（中间层）

对接“第三方生态”的集成层，由社区维护，包含非官方核心合作的第三方工具 / 模型 / 数据源的集成实现（Core 层是抽象，Community 层是这些抽象的具体落地）。

(1) 核心作用

- 低成本对接小众 / 社区级生态：比如阿里百炼（DashScope）、国内大模型（通义千问、文心一言）、开源向量库（FAISS、Chroma）、文档加载器（PyPDFLoader、ExcelLoader）、工具（本地脚本、小众 API）；
- 兼容 Core 层接口：所有集成都遵循 Core 层的抽象，比如 ChatDashScope 实现了 Core 层 BaseChatModel 接口。

（2）关键包安装

```
conda install -c conda-forge langchain-community
```

如果需要对接第三方，安装以下包：

```
conda install -c conda-forge langchain-openai
```

对接阿里百炼平台的 Dashscope：

```
conda install -c conda-forge dashscope
```

3）应用构建层：LangChain / LangGraph（上层应用层）

基于底层抽象 / 集成层封装的“高阶应用组件”，面向应用开发者，无需关注底层接口，直接复用预制的“场景化能力”。

LangChain 的核心是基础 Agent，LangGraph 是 LangChain 生态下的高级工作流编排库，是 LangChain 线性流程能力的补充与升级。

（1）核心作用

- 降低应用开发门槛：封装了常见场景的完整逻辑，比如 RAG（检索增强生成）、智能代理、多轮对话机器人；
- 提供预制模板：比如 RetrievalQA（知识库问答链）、create_openai_tools_agent（工具调用代理）都属于这一层的预制组件。

（2）关键组件

预制 Chains、Agents、RAG 流水线、对话机器人模板。

4）部署运维层 LangServer + LangSmith（运维观测层）

面向生产环境的部署、调试、监控层，解决开发完应用如何上线、如何排查问题的核心需求。

核心组件与功能如表 1.3-1 所示：

表 1.3-1：部署运维层的核心组件与功能

工具	定位	核心功能
LangServer	部署工具	将 LangChain 的 Chains/Agents 封装为 REST API，支持批量调用、流式响应，适配生产部署。
LangSmith	调试 / 监控平台	可视化追踪链 / 代理的执行流程、记录模型调用日志、评估回答质量、调试提示词效果。

2、搜索 + 大模型 + LangChain

今天的 AI 应用已经不再是单一工具，而是搜索引擎 + 大语言模型 + LangChain 框架的协同工作，它们的作用和地位如表 1.3-2 所示：

表 1.3-2：搜索引擎 + 大语言模型 + LangChain 框架的作用和地位

层级	组件	职责
数据层	搜索引擎 + 向量数据库	获取最新信息或检索私有知识库，解决知

		识滞后、无私有数据问题
推理层	大语言模型	理解问题、生成回答、调用工具，即负责语义理解、知识整合、答案生成
编排层	LangChain	串联整个流程：接收问题→检索数据→调用模型→生成答案。管控逻辑、规避幻觉、实现自主决策

这就是“会用工具的大模型”——搜索负责找事实，大模型负责理解和表达，LangChain 负责编排整个流程。

教材未出版，禁止传播，版权所有：柯善斌

1.4 AI 编程入门案例

1.4.1 AI 辅助前端开发：本地记账小工具

通过 Copilot Chat 实现纯 HTML 轻量化工具开发，体验 AI 辅助全流程开发，熟悉需求拆解、代码生成、迭代优化流程，所有数据本地存储、无需联网。

操作步骤如下：

- (1) 创建 aitoools 文件夹（如：E:\ai\aitools）
- (2) 在 VS Code 打开 aitoools 文件夹（作为项目要目录）
- (3) 打开 Copilot Chat，输入以下需求：

帮我做一个纯 HTML 的本地日常开销记账小工具，能输入消费金额、分类、日期，自动统计月度总支出，界面用清爽的浅蓝风格，所有数据存在本地浏览器里不用联网。

- (4) 发送
- (5) 等待代码生成，在生成过程中需要手动“允许”代码审核
- (6) 完成后，保留文件，会在当前项目生成一个.html 文件（如：index.html）。
- (7) 打开文件

进入 aitoools 文件夹，直接打开 index.html（每次生成页面效果不会相同），结果如图 1-4-1 所示：

无需联网，所有记录自动保存在浏览器本地，支持输入金额、分类、日期并统计本月总支出。

消费金额 (元)

例：88.50

分类

请选择分类

日期

2026/07/18

添加记录

本月总支出

120.00 元

记录条目

1

最新分类

餐饮

日期	分类	金额	操作
2026-07-18	餐饮	120.00 元	删除

数据保存在本地浏览器，关闭页面后可在同一浏览器继续查看。

图 1.4-1：本地记账小工具页面

1.4.2 数据可视化实践：农作物数据分析

基于 CSV 原始数据，完成数据清洗、分类统计、可视化图表生成、自动生成分析报告，掌握 AI 辅助数据处理、代码调试、报告输出的完整流程，为后续文本数据预处理奠定实操基础。

操作步骤如下：

- (1) 将文件“2024 年种植业数据.csv”复制到指定文件夹（如：E:\user\data）
- (2) 在控制台安装 pandas

```
conda activate ai_env
conda install pandas
```

- (3) 打开 Copilot Chat，输入以下需求：

本地 **e:\usr\data** 有文件：2024 年种植业数据.csv，现需要做如下操作：

- 1. 数据清洗
 - (1) 删除数值为 0 的数据
 - (2) 删除品类中包含人均的品类
 - (3) 将指标为产量的数值单位中的万吨统一转换为吨
 - (4) 将指标为'播种面积/种植面积'统一转换为转换为种植面积
 - (5) 将指标为种植的数值单位由千公顷转换为亩
- 2. 农作物结构分析
 - (1) 粮食作物通常包括：水稻、小麦、玉米、大豆、薯类、谷物等
 - (2) 经济作物通常包括：油料、棉花、糖料、蔬菜、水果、茶叶、烟叶、药材等
 - (3) 其他作物：饲料、绿肥等用饼状图呈现出各类作物和产量比例和种植面积比例
- 3. 分析报告
 - 帮我制作一份纯 HTML 的分析报告，并对报告进行解读
- 4. 生成 python 源码文件

- (4) 发送
 - (5) 等待代码生成，在生成过程中需要手动“允许”代码审核
 - (6) 完成后，保留文件，会在当前项目生成一个.html 文件（如：analysis_report.html）和源代码文件（如：analyze_crops.py）。同时在 **E:\user\data** 文件夹下生成“种植业分析报告.html”。
 - (7) 打开文件
- 进入 aitools 文件夹，直接打开 analysis_report.html，结果如图 1-4-2 所示：





图 1. 4-3：农作物数据数据代码生成报告

(9) 删除 E:\user\data 文件夹 analysis_report.html，重新运行 analyze_crops.py 程序，查看 E:\user\data 文件夹是生成了 analysis_report.html 文件，打开内容是否相同。

本章小结

本章作为课程入门基石，构建了理论认知-环境搭建-框架入门-实操验证的完整基础体系，核心知识点闭环如下：

- 厘清大语言模型核心原理、特征、分类与能力边界，明确大模型概率生成机制、智能幻觉、知识滞后等原生短板，掌握与传统搜索的核心差异
- 掌握 MaaS 云服务接入模式与国内主流模型生态，熟练掌握 API_KEY 环境变量安全配置规范，规避开发安全风险
- 完成 Conda 虚拟环境、国内镜像、开发工具、交互式编程环境的标准化搭建，形成全书统一可复用的开发环境
- 精准掌握 LangChain 核心定位、六大组件、四层分层生态，建立「搜索+大模型+LangChain」的全局技术架构认知
- 完成入门实操案例，熟悉 AI 辅助开发流程，验证环境可用性，为后续 RAG 检索、智能 Agent、全项目开发奠定基础

教材未出版，禁止传播，版权所有。

课后练习题

一、选择题

- 1、以下不属于 AI 大语言模型（LLM）核心特征的是（）
A. 参数规模庞大 B. 训练数据海量
C. 仅支持单模态处理 D. 具备涌现能力
- 2、按模态划分，能同时处理文本、图像、音频的大模型属于（）
A. 大语言模型 B. 多模态模型 C. 视觉大模型 D. 推理模型
- 3、主流云平台提供的 MaaS 服务不包含以下哪种接入模式（）
A. API 直连 B. 一键部署 C. 本地独立开发 D. 定制微调
- 4、为保护 API_KEY 不被暴露，推荐的做法是（）
A. 直接写在代码中 B. 设置为系统环境变量
C. 保存至文本文档并放在项目根目录 D. 分享至开发群聊
- 5、Conda 与 pip 相比，核心优势不包括（）
A. 仅管理 Python 包 B. 支持多语言包管理
C. 自动处理依赖关系 D. 实现多环境隔离
- 6、创建名为 langchain_env、Python 版本为 3.13 的 conda 环境，正确的命令是（）
A. conda create -n langchain_env python=3.13
B. conda new -n langchain_env python=3.13
C. conda build -n langchain_env python=3.13
D. conda init -n langchain_env python=3.13
- 7、LangChain 的核心定位是（）
A. 一款高性能的大语言模型 B. 连接大模型与各类资源的桥梁
C. 专用于数据存储的向量数据库 D. 大模型训练框架
- 8、负责存储对话历史、让大模型实现多轮对话的 LangChain 组件是（）
A. Models B. Memory C. Indexes D. Prompts
- 9、LangChain 分层生态中，定义了所有上层组件核心抽象接口的是（）
A. LangChain Community B. LangGraph
C. LangChain Core D. LangSmith
- 10、用于 LangChain 应用生产部署，将 Chains/Agents 封装为 REST API 的工具是（）
A. LangServer B. LangSmith C. Jupyter Notebook D. PyCharm

二、简答题

- 1、简述 AI 大语言模型按功能的分类及各类别的核心用途。
- 2、说明使用 conda 创建虚拟环境的核心意义，并列出 conda 环境管理的 5 个常用命令。
- 3、简述 LangChain 的六大核心组件的各自功能。
- 4、阐述 LangChain 分层生态系统的四个层级，并说明每个层级的核心定位与作用。